Functional Graph Discriminative Network for Autism Spectrum Disorder Classification

Rohan Sinha (M25DS008)

Department of Computer Science and Engineering
Indian Institute of Technology Bhilai

Course: Data Science Methods for Brain Functional Connectivity

Outline

1. Objective and Dataset
2. Preprocessing and Tangent Connectivity
3. Graph Construction and FGDN
4. Graph Convolution
5. Experiments and Results
6. Ablation Study
7. Conclusion

Project Objective

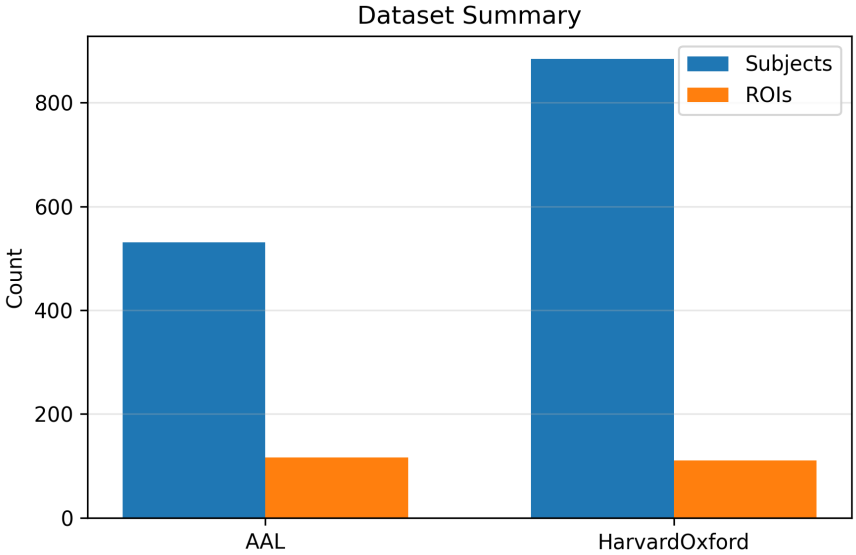
- Reproduce the Functional Graph Discriminative Network (FGDN) approach for ASD classification.
- Use ABIDE preprocessed ROI time-series from AAL and HarvardOxford atlases.
- Convert ROI time-series into tangent functional connectivity matrices.
- Construct ASD and HC class-specific graph templates.
- Train and evaluate unweighted and weighted FGDN models using 5-fold and 10-fold CV.

Dataset

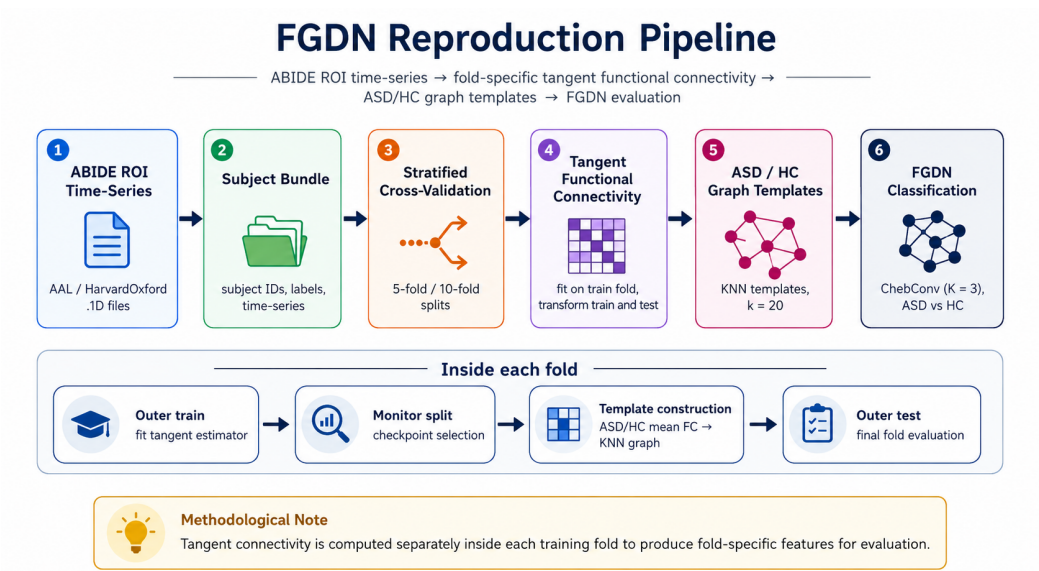
Atlas	Subjects	ROIs
AAL	531	116
HarvardOxford	884	111

- Dataset: ABIDE preprocessed resting-state fMRI ROI time-series.
- Input format: ROI time-series files.
- Label convention: HC $= 0$, ASD $= 1$.

Dataset Summary



Overall Pipeline



Why Tangent Functional Connectivity?

A simple functional connectivity matrix can be computed using correlation:

$$F_i(p, q) = corr(X_i[:, p], X_i[:, q])$$

However, the FGDN pipeline follows a tangent-connectivity representation instead of using only simple correlation.

Reason

Correlation only measures normalized pairwise association between two ROI signals. Tangent connectivity uses covariance matrices, which retain both ROI-to-ROI co-variation and signal variability. Since covariance matrices lie on a positive-definite manifold, they are projected to a common tangent space before classification.

Tangent Connectivity: Starting Point

For each subject:

$$X_i \in \mathbb{R}^{T_i \times N}$$

The aim is to convert this time-series into a tangent functional connectivity matrix:

$$T_i^{tan} \in \mathbb{R}^{N \times N}$$

where:

- For AAL, $N = 116$
- For HarvardOxford, $N = 111$
- T_i^{tan} is later used as the node feature matrix for FGDN

$$X_{\text{graph}} = T_i^{tan}$$

Subject Covariance Matrix

For each subject, first estimate covariance between ROI signals.
If $X_i[:, p]$ and $X_i[:, q]$ are two ROI time-series, then:

$$\Sigma_i^{sample}(p, q) = cov(X_i[:, p], X_i[:, q])$$

So the sample covariance matrix is:

$$\Sigma_i^{sample} \in \mathbb{R}^{N \times N}$$

- Diagonal entries: variance of individual ROI signals.
- Off-diagonal entries: covariance between ROI pairs.
- Σ_i^{sample} summarizes subject-specific brain connectivity structure.

Reference Covariance: Geometric Mean

Tangent connectivity needs one common reference covariance for the training group.
Given training-subject covariance matrices:

$$\Sigma_1, \Sigma_2, \ldots, \Sigma_m$$

the reference covariance is the geometric mean:

$$\bar{\Sigma} = \arg\min_{\Sigma} \sum_{i=1}^{m} d_R^2(\Sigma, \Sigma_i)$$

where the Riemannian distance is:

$$d_R(A, B) = \left\| \log\left(A^{-1/2} B A^{-1/2}\right) \right\|_F$$

- $\bar{\Sigma}$ is the central covariance structure of the training subjects.
- It is not a simple arithmetic mean.

Projection to Tangent Space

After estimating the reference covariance $\bar{\Sigma}$, each subject covariance matrix is projected to the tangent space:

$$T_i^{tan} = \log\left(\bar{\Sigma}^{-1/2}\Sigma_i\bar{\Sigma}^{-1/2}\right)$$

where:

- Σ_i = Covariance of subject i
- $\bar{\Sigma}$ = geometric mean covariance of training subjects
- T_i^{tan} = tangent functional connectivity matrix

In my FGDN pipeline:

$$X_i = T_i^{tan}$$

Each row of T_i^{tan} becomes the feature vector of one ROI node.

Graph Representation

A graph is defined as:

$$G = (V, E, W)$$

In this project:

- Node = brain ROI
- Edge = template connection between ROIs
- Node feature = one row of the subject tangent FC matrix
- Graph templates = ASD template and HC template

For each subject:

$$G_{ASD} = (T_i^{tan}, A_{ASD}), \qquad G_{HC} = (T_i^{tan}, A_{HC})$$

ASD and HC Graph Templates

Class mean connectivity matrices are computed from training subjects:

$$\bar{F}_{ASD} = \frac{1}{N_{ASD}} \sum_{i:y_i=1} F_i$$

$$\bar{F}_{HC} = \frac{1}{N_{HC}} \sum_{i:y_i=0} F_i$$

Then KNN graph construction gives:

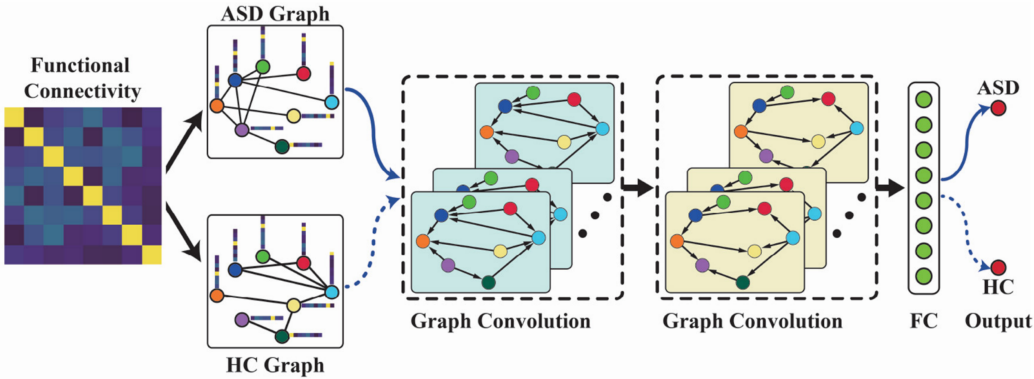
$$A_{ASD}, A_{HC}$$

with:

$$k = 20$$

- ASD branch uses A_{ASD}
- HC branch uses A_{HC}

FGDN Architecture



Basic GCN: Self-Loops and Degree Matrix

For a graph with adjacency matrix A, first add self-loops:

$$\hat{A} = A + I_N$$

Then compute the degree matrix:

$$\hat{D}_{ii} = \sum_j \hat{A}_{ij}$$

- $\hat{A}$ allows each node to keep its own information.
- $\hat{D}_{ii}$ counts the total connection strength of node i.
- This prevents high-degree nodes from dominating aggregation.

Basic GCN: Normalized Propagation

Normalize the adjacency matrix:

$$\tilde{A} = \hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}}$$

Then a basic GCN layer is:

$$H = \sigma(\tilde{A} X W)$$

- X: input node feature matrix
- W: learnable weight matrix
- $\tilde{A}$: normalized adjacency matrix
- H: updated node features

Meaning: each node aggregates normalized information from itself and its neighbours.

Spectral Graph Convolution

For graph adjacency/weight matrix W, define degree matrix:

$$D_{ii} = \sum_j W_{ij}$$

Normalized graph Laplacian:

$$L = I_N - D^{-\frac{1}{2}} W D^{-\frac{1}{2}}$$

Eigen-decomposition:

$$L = U \Lambda U^T$$

- U: graph Fourier basis
- Λ: graph frequencies
- L: captures graph structure

Spectral Graph Convolution

For a graph signal x, spectral convolution is:

$$g_\theta *_G x = U g_\theta(\Lambda) U^T x$$

Step-wise interpretation:

$$U^T x \quad \rightarrow \quad g_\theta(\Lambda) \quad \rightarrow \quad U$$

- $U^T x$: transform signal to graph-frequency domain.
- $g_\theta(\Lambda)$: apply learnable spectral filter.
- U: transform back to node domain.

Problem: computing U requires expensive eigendecomposition.

Chebyshev Graph Convolution

ChebConv avoids explicit eigendecomposition by approximating the spectral filter:

$$g_\theta *_G x \approx \sum_{k=0}^{K-1} \theta_k T_k(\tilde{L})x$$

where:

$$\tilde{L} = \frac{2L}{\lambda_{\max}} - I_N$$

- $T_k(\tilde{L})$: Chebyshev polynomial
- θ_k: learnable coefficient
- K: filter order

ChebConv performs localized graph filtering without computing U.

ChebConv in FGDN Model

Chebyshev recurrence:

$$T_0(\tilde{L}) = I_N, \qquad T_1(\tilde{L}) = \tilde{L}$$

$$T_k(\tilde{L}) = 2\tilde{L}T_{k-1}(\tilde{L}) - T_{k-2}(\tilde{L})$$

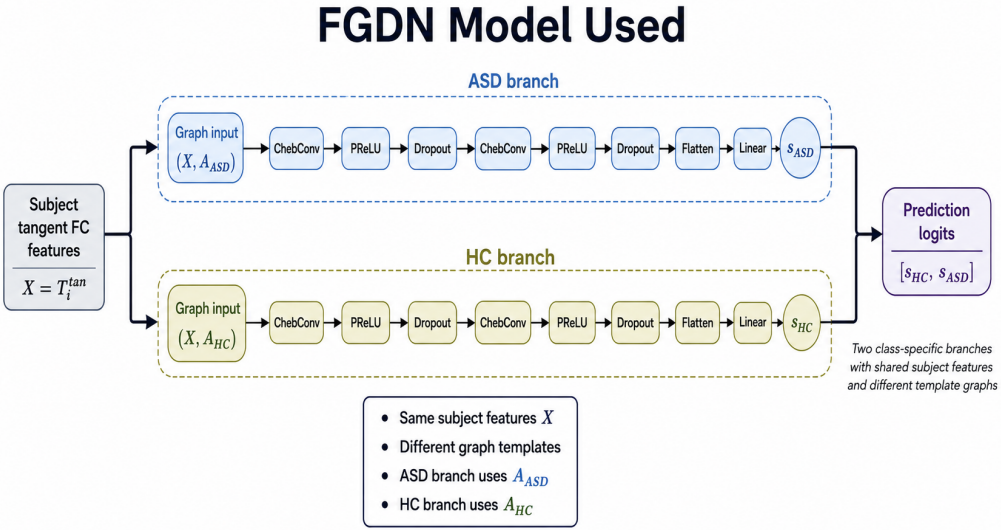
In my baseline FGDN model:

$$K = 3$$

$$H = T_0(\tilde{L})X\Theta_0 + T_1(\tilde{L})X\Theta_1 + T_2(\tilde{L})X\Theta_2$$

This combines self-information, direct-neighbour information, and wider local graph-neighbour information.

FGDN Model Used



Weighted FGDN

Unweighted FGDN uses binary graph edges:

$$A_{ij} \in \{0, 1\}$$

Weighted FGDN uses edge weights:

$$w_{ij} = \exp\left(-\frac{d(i,j)^2}{2\theta^2}\right)$$

where:

- $d(i,j)$ = distance between ROI feature vectors
- θ = scale parameter estimated from KNN distances

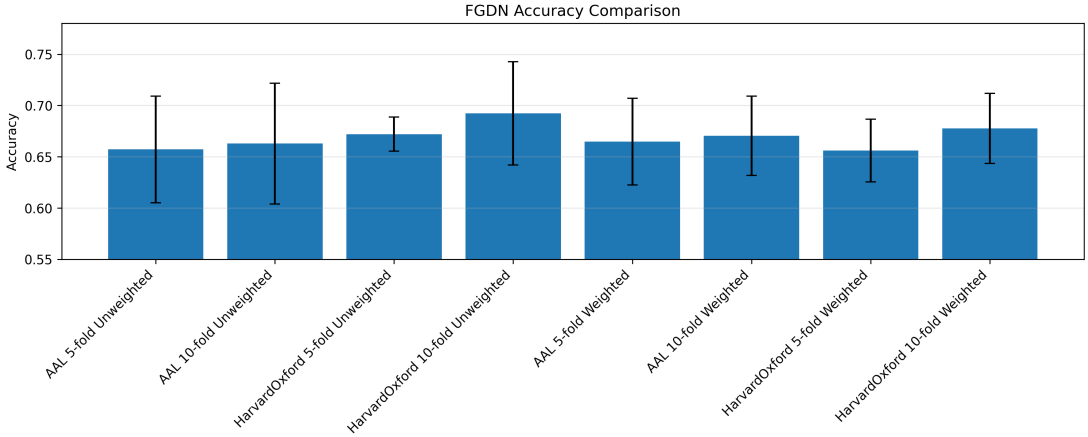
Purpose

Weighted edges preserve connection strength instead of using only binary connectivity.

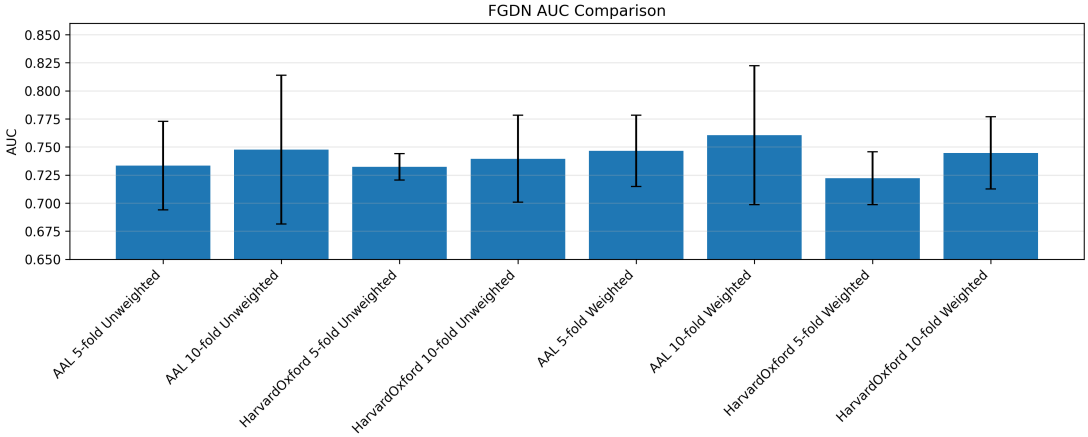
Experimental Setup

Parameter	Baseline Value
KNN neighbours	20
Chebyshev order	3
Hidden channels	64
Dropout	0.1
Batch size	16
Learning rate	0.0001
Weight decay	0.0005
Optimizer	Adam
CV settings	5-fold, 10-fold

Accuracy Results



AUC Results


FGDN AUC Comparison
AUC
0.850
0.825
0.800
0.775
0.750
0.725
0.700
0.675
0.650
AAL 5-fold Unweighted
AAL 10-fold Unweighted
HarvardOxford 5-fold Unweighted
HarvardOxford 10-fold Unweighted
AAL 5-fold Weighted
AAL 10-fold Weighted
HarvardOxford 5-fold Weighted
HarvardOxford 10-fold Weighted

Final Result Summary

Variant	Setting	Acc.	AUC
Unweighted	AAL 5-fold	0.6572	0.7333
Unweighted	AAL 10-fold	0.6629	0.7476
Unweighted	HO 5-fold	0.6720	0.7324
Unweighted	HO 10-fold	**0.6924**	0.7396
Weighted	AAL 5-fold	0.6648	0.7466
Weighted	AAL 10-fold	0.6703	**0.7606**
Weighted	HO 5-fold	0.6561	0.7222
Weighted	HO 10-fold	0.6776	0.7447

Comparison with Original FGDN Paper

Source	Setting	CV	Acc.	AUC
Paper	FGDN HO Tangent	5-fold	0.714	0.769
Ours	HO Unweighted Tangent	5-fold	0.672	0.732
Ours	HO Weighted Tangent	5-fold	0.656	0.722
Paper	FGDN AAL Tangent	5-fold	0.723	0.751
Ours	AAL Unweighted Tangent	5-fold	0.657	0.733
Ours	AAL Weighted Tangent	5-fold	0.665	0.747
Paper	FGDN HO Tangent	10-fold	0.714	0.777
Ours	HO Unweighted Tangent	10-fold	0.692	0.740
Ours	HO Weighted Tangent	10-fold	0.678	0.745
Paper	FGDN AAL Tangent	10-fold	0.708	0.794
Ours	AAL Unweighted Tangent	10-fold	0.663	0.748
Ours	AAL Weighted Tangent	10-fold	0.670	0.761

Ablation Study Design

The ablation study was done on the AAL tangent setting.

1. **Exploratory stage:** AAL 5-fold fold 1.
2. **Confirmation stage:** selected candidates on full AAL 5-fold CV.
3. **Final stage:** top two candidates on full AAL 10-fold CV.

Ablation group	Tested variable
Graph sparsity	template_k
Spectral filter	cheb_k
Model capacity	hidden channels
Regularization	dropout, weight decay
Optimization	learning rate, batch size
Graph weighting	binary vs weighted template

Fold-1 Exploratory Ablation

Group	Best setting	Acc.	AUC
KNN neighbours	template_k = 20	0.6729	**0.7535**
Dropout	dropout = 0.0	0.6822	0.7521
Hidden channels	hidden = 128	**0.7196**	0.7486
Chebyshev order	cheb_k = 1	0.6636	0.7472
Weighted vs binary	binary	0.6449	0.7409
Weight decay	0.0005	0.6916	0.7399
Learning rate	0.00005	0.6916	0.7399
Batch size	16	0.6636	0.7353

AAL 5-Fold Confirmation Ablation

Candidate	Acc. mean	Acc. SD	AUC mean	AUC SD
template_k = 10	0.6554	0.0445	**0.7511**	0.0426
hidden = 128	**0.6742**	0.0466	0.7506	0.0520
combined top	0.6723	0.0573	0.7504	0.0504
cheb_k = 1	0.6610	0.0715	0.7472	0.0546
dropout = 0.0	0.6535	0.0461	0.7458	0.0634

Observation

template_k = 10 gave the best AUC, while hidden = 128 gave the best accuracy.

Final AAL 10-Fold Ablation

Setting	Acc. mean	Acc. SD	AUC mean	AUC SD
Baseline: hidden=64, k=20	0.6629	0.0589	0.7476	0.0663
template_k = 10	**0.6741**	0.0759	0.7461	0.0769
hidden = 128	0.6722	0.0734	**0.7568**	0.0909

Final ablation result

hidden = 128 improved AUC from **0.7476** to **0.7568**.

Ablation Study Conclusion

- The best final AUC was obtained by increasing hidden channels from 64 to 128.
- template_k = 10 gave the highest accuracy, but did not improve AUC over the baseline.
- The combined top setting did not outperform simpler single-factor changes.
- Therefore, the most useful ablation improvement was increasing model capacity.

Selected final ablation setting

hidden_channels = 128

Conclusion

- I reproduced FGDN for ASD classification using ABIDE ROI time-series.
- Tangent functional connectivity was used as graph node features.
- ASD and HC graph templates were constructed using KNN.
- Chebyshev graph convolution was used for graph learning.
- Weighted FGDN achieved the best overall AUC on AAL 10-fold CV.
- The ablation study showed that hidden_channels = 128 improved AAL 10-fold AUC from 0.7476 to 0.7568.

Thank You

Questions?